

Matmul in CUDA

Ahmad Sharif

ahmadsharif@gmail.com

2026/2/29

About me

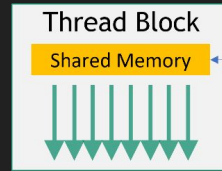
- Ahmad Sharif
- C++ & CUDA user
- Performance enthusiast
- I like debugging and fixing slowness
- Currently work at Meta on performance
 - Not representing Meta today



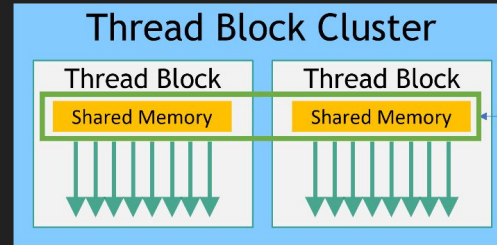
Agenda

- CUDA programming model
- H100 architecture
- Matmul case study

Per thread registers and
local memory

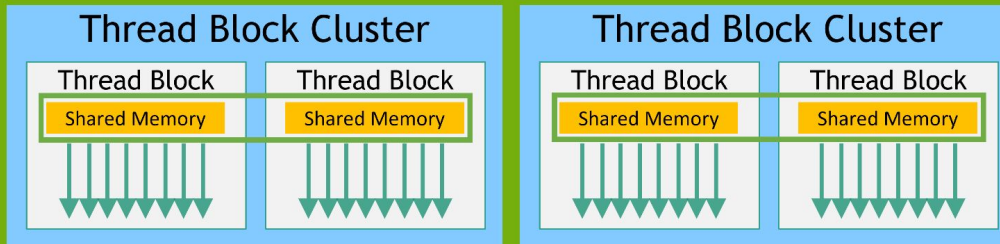


Per block Shared memory



Shared memory of all
thread blocks in a cluster
form Distributed Shared
Memory

Grid with Clusters



Global Memory

Global Memory shared
between all GPU kernels

HOPPER H100 TENSOR CORE GPU

80B Transistors, TSMC 4N



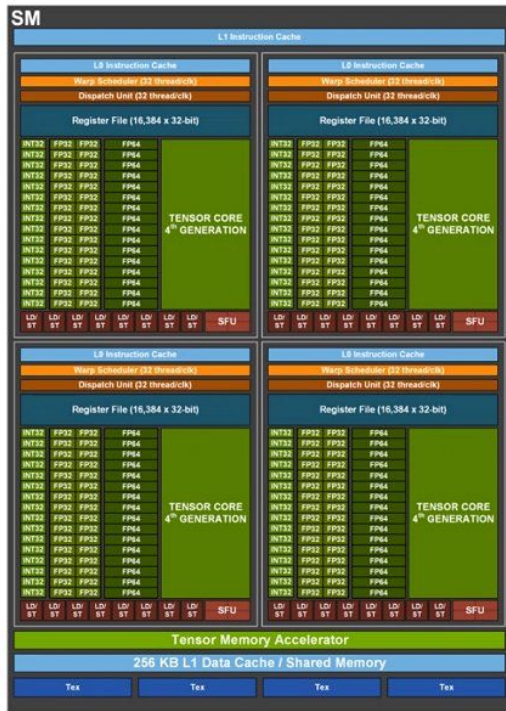
2nd Gen Multi-Instance GPU
Confidential Computing
PCIe Gen5

New Memory System
World's First HBM3 DRAM
Larger 50MB L2

132 SMs
2x Performance per Clock
4th Gen Tensor Core
Thread Block Clusters

4th Gen NVLink
900 GB/s total bandwidth
New SHARP support
NVLink Network

NEW HOPPER SM ARCHITECTURE



- New 4th Gen Tensor Core
- 2x faster FP32 & FP64 FMA
- New DPX instruction set
- New Thread Block Clusters
 - Turn locality into efficiency
- New Tensor Memory Accelerator
 - Fully asynchronous data movement
- 256 KB L1\$ / Shared Memory

H100 numbers

- Clock=1.98GHz
- SMs per GPU=132
- Sub-partitions per SM=4
- FP32 units per sub-partition=32 (= 1 warp)
- FP32 flops per SM per cycle= $4 \times 32 \times 2 = 256$
- FP32 TFlops per GPU per second= $256 \times 132 \times 1.98e9 / 1e12$
 - 67TFlops
- # tensor cores per sub-partition=4
- flops per cycle per tensor core= 256×2 (for FMA)
- TF32 flops per SM per cycle= $4 \times 256 \times 2 = 2048$
- TF32 TFlops per GPU per second= $2048 \times 132 \times 1.83e9 / 1e12$
 - 494 TFlops
- Bandwidth=3.35 TB/s

Technical Specifications

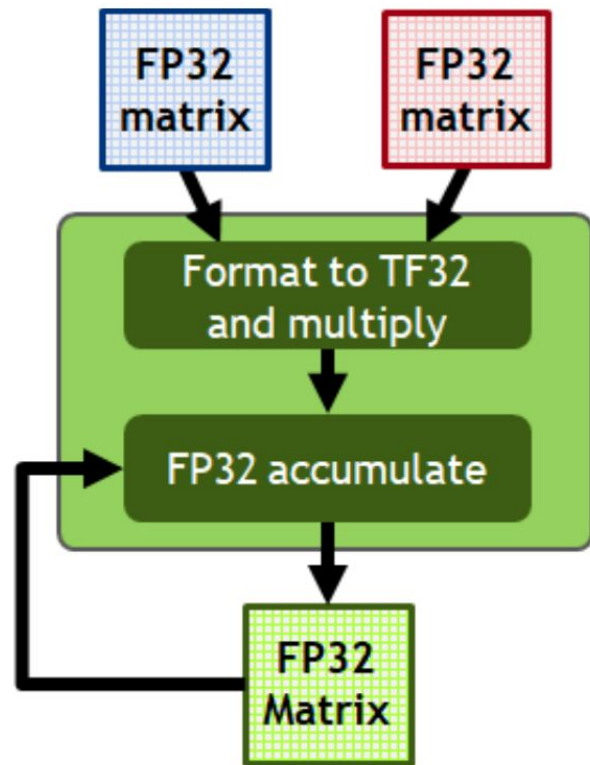
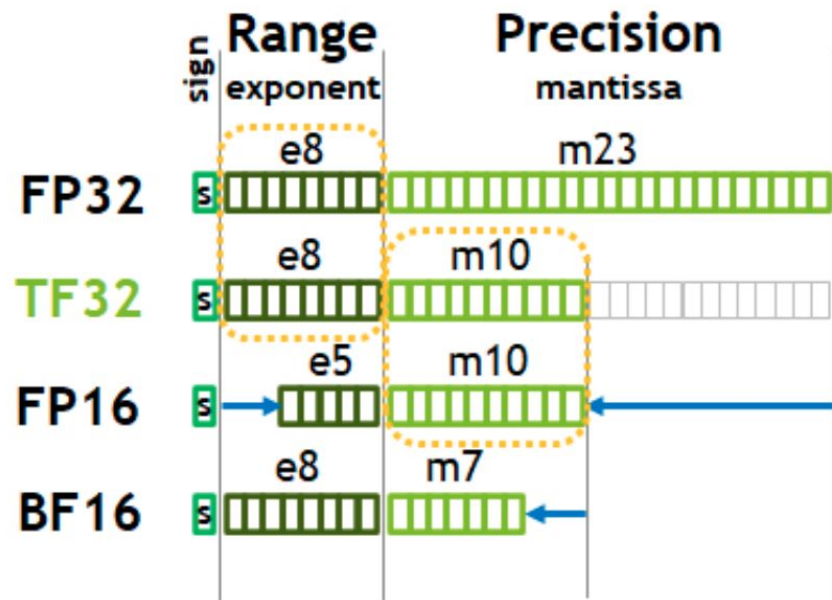
	H100 SXM	H100 NVL
FP64	34 teraFLOPS	30 teraFLOPS
FP64 Tensor Core	67 teraFLOPS	60 teraFLOPS
FP32	67 teraFLOPS	60 teraFLOPS
TF32 Tensor Core*	989 teraFLOPS	835 teraFLOPS
BFLOAT16 Tensor Core*	1,979 teraFLOPS	1,671 teraFLOPS
FP16 Tensor Core*	1,979 teraFLOPS	1,671 teraFLOPS
FP8 Tensor Core*	3,958 teraFLOPS	3,341 teraFLOPS
INT8 Tensor Core*	3,958 TOPS	3,341 TOPS
GPU Memory	80GB	94GB
GPU Memory Bandwidth	3.35TB/s	3.9TB/s
Decoders	7 NVDEC 7 JPEG	7 NVDEC 7 JPEG
Max Thermal Design Power (TDP)	Up to 700W (configurable)	350-400W (configurable)
Multi-Instance GPUs	Up to 7 MIGs @ 10GB each	Up to 7 MIGS @ 12GB each
Form Factor	SXM	PCIe dual-slot air-cooled
Interconnect	NVIDIA NVLink™: 900GB/s PCIe Gen5: 128GB/s	NVIDIA NVLink: 600GB/s PCIe Gen5: 128GB/s
Server Options	NVIDIA HGX H100 Partner and NVIDIA-Certified Systems™ with 4 or 8 GPUs NVIDIA DGX H100 with 8 GPUs	Partner and NVIDIA-Certified Systems with 1-8 GPUs
NVIDIA Enterprise	Add-on	Included

*With sparsity

Matmul

- Why matmul?
 - linear layer
 - attention
 - backward pass too
- Interesting case study
- $C = A \times B$
 - $[4k \times 8k] = [4k \times 16k] * [16k \times 8k]$
 - A bytes = 256MB
 - B bytes = 512MB
 - C bytes = 128MB
 - FMAs (FLOPs) = $2 * 4k * 8k * 16k = 1\text{TFLOP}$
 - FLOPs : bytes ratio > 1000 FLOPs per byte

Kernel	Duration (ms)	TFLOP/S	%age of cublas FP32	%age of cublas TF32
cuBLAS (FP32)	30.96	35.51	100%	11%
cuBLAS (TF32)	3.46	317.78	895%	100%



TensorFloat-32 (TF32) provides the range of FP32 with the precision of FP16, 8x precision vs. BF16 (left). A100 accelerates tensor math with TF32 while supporting FP32 input and output data (right), enabling easy integration into DL and HPC programs and automatic acceleration of DL frameworks.

Figure 9. TensorFloat-32 (TF32)

Simple kernel

1. Launch one thread per output element of C
2. Compute the dot product for that element
3. Write it out to C

$M=4096$

$N=8192$

$K=16384$

bytes per element=4 (Float32)

Each thread computes one result

Each thread reads $2 * K$ elements

Total of $M*N$ results

Total bytes read = $M * N * K * 2 * \text{sizeof}(\text{Float32})$
= 4.4TB

$K=16384$

$N=8192$

single
column

B

$K=16384$

$M=4096$

single row

A

one
result

C

matmul_naive

```
// Naive: one thread per output element

__global__ void matmul_naive(const float* A, const float* B, float* C, int M, int K, int N) {
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    if (row >= M || col >= N) return;

    float sum = 0.0f;
    for (int i = 0; i < K; i++) {
        sum += A[row * K + i] * B[i * N + col];
    }
    C[row * N + col] = sum;
}
```

matmul_naive host

```
// Kernel configs  
dim3 block(16, 16);  
dim3 grid((N + block.x - 1) / block.x, (M + block.y - 1) / block.y);  
  
matmul_naive<<<grid, block>>>(d_A, d_B, d_C, M, K, N);
```

Kernel	Duration (ms)	TFLOP/S	%age of cublas FP32	%age of cublas TF32
cuBLAS (FP32)	30.96	35.51	100%	11%
cuBLAS (TF32)	3.46	317.78	895%	100%
matmul_naive	296.24	3.71	10%	1%

Tiled matmul

1. Load a tile of A ($TILE_SIZE \times TILE_SIZE$) to SMEM
2. Load a tile of B ($TILE_SIZE \times TILE_SIZE$) to SMEM
3. Update C in a register
4. Repeat 1-3 for the K dimension
5. Write result to GMEM

M=4096

N=8192

K=16384

bytes per element=4 (Float32)

$TILE_SIZE = T = 32$

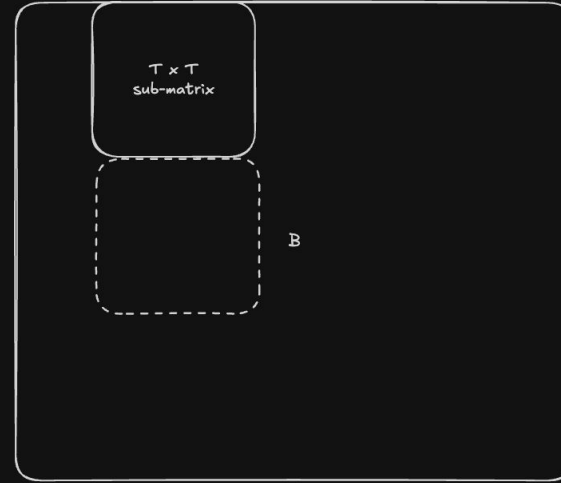
Total bytes read = $T * T * 2 * \text{sizeof}(\text{Float32}) * (K/T) * (M/T) * (N/T)$
 $= 2 * 4 * M * N * K / T$
 $= 137\text{GB}$

32x savings in theory

Less in practice because L2 already does caching for naive

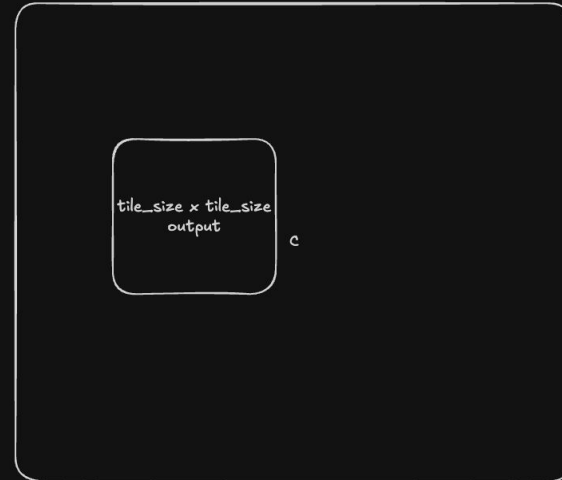
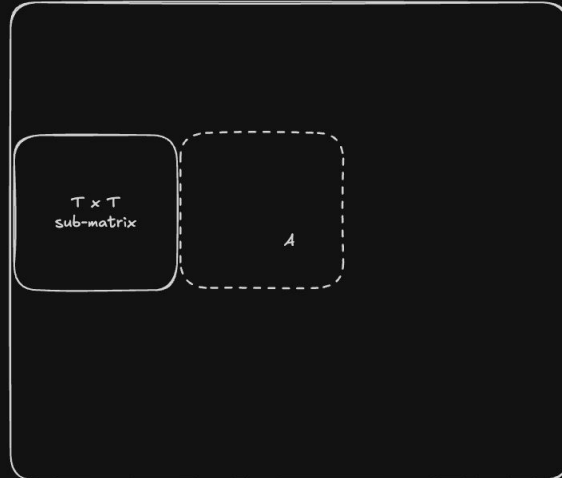
K=16384

N=8192



K=16384

M=4096



SMEM Tiled Matmul Details

Tile size is 32x32. Why? Threads per block limit

2 SMEM loads per FMA

SM can load 32 FP32 elements per cycle from SMEM

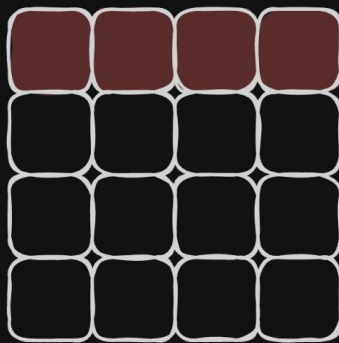
SM can execute 128 FP32 FMAs per cycle

FMA : SMEM ratio = 4 : 1

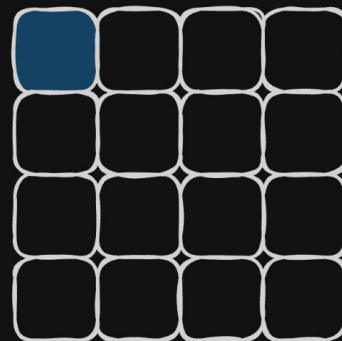
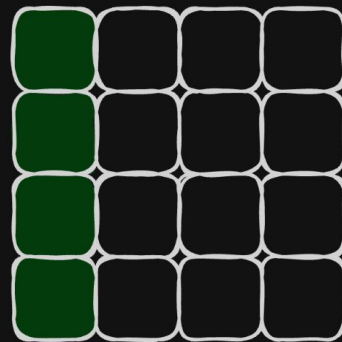
Our kernel does 1 : 2

8x difference in "arithmetic intensity"

A tile in SMEM



B tile in SMEM



C in registers
(1 element per thread)

matmul_tiled

```
// Shared memory tiled GEMM: load BLK*BLK tiles of A and B into smem,  
// compute partial dot products, slide along K.  Pure FP32, no tensor cores.  
template <int BLK>  
__global__ void matmul_tiled(const float* A, const float* B, float* C, int M, int K, int N) {  
    __shared__ float As[BLK][BLK];  
    __shared__ float Bs[BLK][BLK];  
  
    int row = blockIdx.y * BLK + threadIdx.y;  
    int col = blockIdx.x * BLK + threadIdx.x;  
  
    float sum = 0.0f;  
    for (int t = 0; t < K; t += BLK) {  
        // Collaborative load: each thread loads one element of each tile  
        As[threadIdx.y][threadIdx.x] = (row < M && t + threadIdx.x < K)  
            ? A[row * K + t + threadIdx.x] : 0.0f;  
        Bs[threadIdx.y][threadIdx.x] = (t + threadIdx.y < K && col < N)  
            ? B[(t + threadIdx.y) * N + col] : 0.0f;  
  
        __syncthreads();  
  
        for (int i = 0; i < BLK; i++)  
            sum += As[threadIdx.y][i] * Bs[i][threadIdx.x];  
        __syncthreads();  
    }  
  
    if (row < M && col < N)  
        C[row * N + col] = sum;  
}
```

Kernel	Duration (ms)	TFLOP/S	%age of cublas FP32	%age of cublas TF32
cuBLAS (FP32)	30.96	35.51	100%	11%
cuBLAS (TF32)	3.46	317.78	895%	100%
matmul_naive	296.24	3.71	10%	1%
matmul_tiled<32>	176.78	6.22	18%	2%

Register Tiling

1. Load a tile of A and B to SMEM
2. Load a COLUMN of A to REGISTERS
3. Load a ROW of B to REGISTERS
4. Compute the OUTER PRODUCT and update C in REGISTERS

A_tile = 16×8 in SMEM

B_tile = 8×16 in SMEM

A_slice = 8×1 in REGS

B_slice = 1×8 in REGS

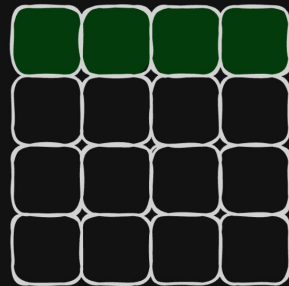
C_regs = 8×8 in REGS

16 loads and 64 FMAs

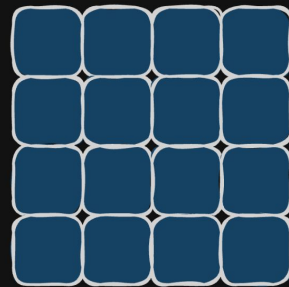
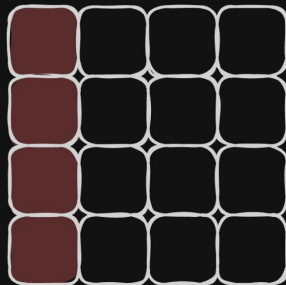
1 : 4 LOAD to FMA

=> 8x higher arithmetic intensity

B tile in SMEM



A tile in SMEM



C in registers
(1 element per thread)

matmul_tiled_reg

```
// Register-tiled GEMM: each thread computes a TM×TN sub-block of outputs.
// The outer-product inner loop does TM+TN smem loads for TM×TN FMAs,
// dramatically improving the compute-to-load ratio vs the basic tiled kernel.
template <int BLK_M, int BLK_N, int BLK_K, int TM, int TN>
__global__ void matmul_tiled_reg(const float* A, const float* B, float* C, int M, int K, int N) {
    __shared__ float As[BLK_M][BLK_K];
    __shared__ float Bs[BLK_K][BLK_N];

    // Accumulator registers
    float accum[TM][TN] = {};

    // Number of float elements each thread must load to cooperatively fill smem
    constexpr int BLOCK_SIZE = THREADS_M * THREADS_N; // 256
    constexpr int A_TILE_ELEMS = BLK_M * BLK_K; // 128*8 = 1024
    constexpr int B_TILE_ELEMS = BLK_K * BLK_N; // 8*128 = 1024
    constexpr int A_LOADS_PER_THREAD = A_TILE_ELEMS / BLOCK_SIZE; // 4
    constexpr int B_LOADS_PER_THREAD = B_TILE_ELEMS / BLOCK_SIZE; // 4

    for (int t = 0; t < K; t += BLK_K) {
        // Cooperative load of A tile into shared memory
        // Cooperative load of B tile into shared memory
        __syncthreads();
        // Outer-product accumulation: TM+TN loads → TM×TN FMAs per k step
        __syncthreads();
    }

    // Store results
}
```

Kernel	Duration (ms)	TFLOP/S	%age of cublas FP32	%age of cublas TF32
cuBLAS (FP32)	30.96	35.51	100%	11%
cuBLAS (TF32)	3.46	317.78	895%	100%
matmul_naive	296.24	3.71	10%	1%
matmul_tiled<32>	176.78	6.22	18%	2%
matmul_tiled_reg	67.7	16.24	46%	5%

matmul_tiled_vec

```
// Register-tiled GEMM: each thread computes a TM×TN sub-block of outputs.
// The outer-product inner loop does TM+TN smem loads for TM×TN FMAs,
// dramatically improving the compute-to-load ratio vs the basic tiled kernel.
template <int BLK_M, int BLK_N, int BLK_K, int TM, int TN>
__global__ void matmul_tiled_reg(const float* A, const float* B, float* C, int M, int K, int N) {
    __shared__ float As[BLK_M][BLK_K + PAD];
    __shared__ float Bs[BLK_K][BLK_N + PAD];

    // Accumulator registers
    float accum[TM][TN] = {};

    for (int t = 0; t < K; t += BLK_K) {
        // Cooperative load of A tile into shared memory
        float4 val = reinterpret_cast<const float4*>(&A[global_row * K + global_col])[0];
        As[a_row][a_col] = val.x;
        As[a_row][a_col + 1] = val.y;
        As[a_row][a_col + 2] = val.z;
        As[a_row][a_col + 3] = val.w;

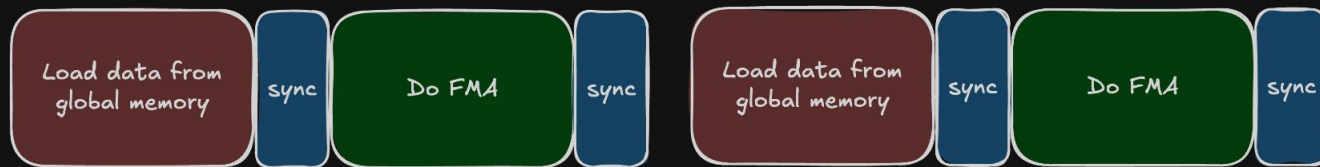
        // Cooperative load of B tile into shared memory
        __syncthreads();
        // Outer-product accumulation: TM+TN loads → TM×TN FMAs per k step
        __syncthreads();
    }

    // Store results
}
```

Kernel	Duration (ms)	TFLOP/S	%age of cublas FP32	%age of cublas TF32
cuBLAS (FP32)	30.96	35.51	100%	11%
cuBLAS (TF32)	3.46	317.78	895%	100%
matmul_naive	296.24	3.71	10%	1%
matmul_tiled<32>	176.78	6.22	18%	2%
matmul_tiled_reg	67.7	16.24	46%	5%
matmul_tiled_vec	50.42	21.81	61%	7%

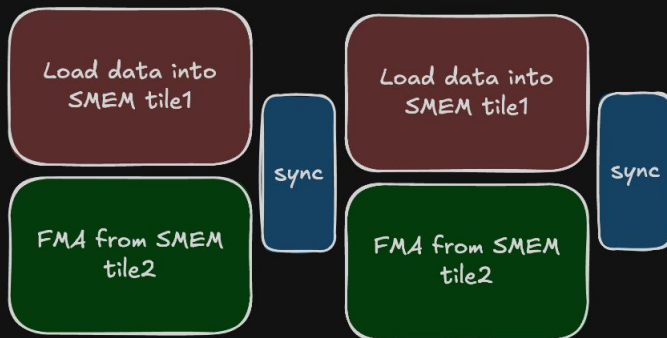
Time →

Old kernel does not overlap data loading with math ops



math HW is
idle here for
this block

load HW is
idle here for
this block



Pipe kernel tries to ensure load and math HW are both busy for the same block

matmul_tiled_pipe

```
// Register-tiled GEMM with vectorized loads and double-buffered shared memory.
// Overlaps global memory loads of the next K-tile with compute on the current tile.
template <int BLK_M, int BLK_N, int BLK_K, int TM, int TN>
__global__ void matmul_tiled_pipe(const float* A, const float* B, float* C, int M, int K, int N) {
    constexpr int PAD = 4;
    __shared__ float As[2][BLK_M][BLK_K + PAD];
    __shared__ float Bs[2][BLK_K][BLK_N + PAD];

    for (int tile = 0; tile < num_tiles; tile++) {
        int cur_buf = tile & 1;
        int nxt_buf = 1 - cur_buf;

        // Start loading next tile into the other buffer
        if (tile + 1 < num_tiles) {
            load_A_tile((tile + 1) * BLK_K, nxt_buf);
            load_B_tile((tile + 1) * BLK_K, nxt_buf);
        }

        // Compute on current buffer (loads and compute overlap at the warp level
        // because loads go through the memory pipeline while FMAs use the math pipe)
        compute_tile(cur_buf);

        __syncthreads();
    }
}
```

Kernel	Duration (ms)	TFLOP/S	%age of cublas FP32	%age of cublas TF32
cuBLAS (FP32)	30.96	35.51	100%	11%
cuBLAS (TF32)	3.46	317.78	895%	100%
matmul_naive	296.24	3.71	10%	1%
matmul_tiled<32>	176.78	6.22	18%	2%
matmul_tiled_reg	67.7	16.24	46%	5%
matmul_tiled_vec	50.42	21.81	61%	7%
matmul_tiled_pipe	73.32	15	42%	5%

vec vs pipe

Section: Launch Statistics

Metric Name	Metric Unit	Metric Value
Block Size		256
Cluster Scheduling Policy		PolicySpread
Cluster Size		0
Function Cache Configuration		CachePreferNone
Grid Size		2,048
Registers Per Thread	register/thread	128
Shared Memory Configuration Size	Kbyte	65.54
Driver Shared Memory Per Block	Kbyte/block	1.02
Dynamic Shared Memory Per Block	byte/block	0
Static Shared Memory Per Block	Kbyte/block	10.37
# SMs	SM	132
Stack Size		1,024
Threads	thread	524,288
# TPCs		66
Enabled TPC IDs		all
Uses Green Context		0
Waves Per SM		7.76

Section: Launch Statistics

Metric Name	Metric Unit	Metric Value
Block Size		256
Cluster Scheduling Policy		PolicySpread
Cluster Size		0
Function Cache Configuration		CachePreferNone
Grid Size		2,048
Registers Per Thread	register/thread	130
Shared Memory Configuration Size	Kbyte	32.77
Driver Shared Memory Per Block	Kbyte/block	1.02
Dynamic Shared Memory Per Block	byte/block	0
Static Shared Memory Per Block	Kbyte/block	20.74
# SMs	SM	132
Stack Size		1,024
Threads	thread	524,288
# TPCs		66
Enabled TPC IDs		all
Uses Green Context		0
Waves Per SM		15.52

vec vs pipe

Section: Occupancy

Metric Name	Metric Unit	Metric Value
Max Active Clusters	cluster	0
Max Cluster Size	block	8
Overall GPU Occupancy	%	0
Cluster Occupancy	%	0
Block Limit Barriers	block	32
Block Limit SM	block	32
Block Limit Registers	block	2
Block Limit Shared Mem	block	5
Block Limit Warps	block	8
Theoretical Active Warps per SM	warp	16
Theoretical Occupancy	%	25
Achieved Occupancy	%	24.30
Achieved Active Warps Per SM	warp	15.55

Section: Occupancy

Metric Name	Metric Unit	Metric Value
Max Active Clusters	cluster	0
Max Cluster Size	block	8
Overall GPU Occupancy	%	0
Cluster Occupancy	%	0
Block Limit Barriers	block	32
Block Limit SM	block	32
Block Limit Registers	block	1
Block Limit Shared Mem	block	1
Block Limit Warps	block	8
Theoretical Active Warps per SM	warp	8
Theoretical Occupancy	%	12.50
Achieved Occupancy	%	12.50
Achieved Active Warps Per SM	warp	8.00

matmul_cute_simple

```
// CuTe MMA kernel: global memory -> registers -> MMA (no shared memory)
// Follows the CuTe GEMM tutorial pattern (sgemm_1).
template<int BLK_M, int BLK_N, int BLK_K, class TiledMMA>
__global__ void matmul_cute_simple(const float* A_ptr, const float* B_ptr, float* C_ptr,
                                  int M, int K, int N, TiledMMA tiled_mma) {

    // Global memory tensors
    // A: (M, K) row-major
    auto mA = make_tensor(make_gmem_ptr(A_ptr), make_shape(M, K), make_stride(K, Int<1>{}));
    // B: stored (K,N) row-major, viewed as (N,K) for the TN MMA
    auto mB = make_tensor(make_gmem_ptr(B_ptr), make_shape(N, K), make_stride(Int<1>{}, N));
    // C: (M, N) row-major
    auto mC = make_tensor(make_gmem_ptr(C_ptr), make_shape(M, N), make_stride(N, Int<1>{}));

    // CTA tiling: blockIdx.x -> M, blockIdx.y -> N
    auto gA = local_tile(mA, make_shape(Int<BLK_M>{}, Int<BLK_K>{}),
                        make_coord(blockIdx.x, _)); // (BLK_M, BLK_K, k)
    auto gB = local_tile(mB, make_shape(Int<BLK_N>{}, Int<BLK_K>{}),
                        make_coord(blockIdx.y, _)); // (BLK_N, BLK_K, k)
    auto gC = local_tile(mC, make_shape(Int<BLK_M>{}, Int<BLK_N>{}),
                        make_coord(blockIdx.x, blockIdx.y)); // (BLK_M, BLK_N)

    // Per-thread MMA partitions
    auto thr_mma = tiled_mma.get_thread_slice(threadIdx.x);
    auto tCgC = thr_mma.partition_C(gC); // (MMA, MMA_M, MMA_N)
    auto tCrC = thr_mma.partition_fragment_C(gC); // accumulator in registers
    clear(tCrC);

    auto tCrA = thr_mma.partition_fragment_A(gA(_, _, 0)); // A fragment in registers
    auto tCrB = thr_mma.partition_fragment_B(gB(_, _, 0)); // B fragment in registers

    // Main loop over K
    int k_tile_max = size<2>(gA);
    for (int k = 0; k < k_tile_max; k++) {
        copy(thr_mma.partition_A(gA(_, _, k)), tCrA);
        copy(thr_mma.partition_B(gB(_, _, k)), tCrB);
        gemm(tiled_mma, tCrA, tCrB, tCrC);
    }

    // Store result
    copy(tCrC, tCgC);
}
```

	0	1	2	3	4	5	6	7
0	T0 V0	T4 V0	T8 V0	T12 V0	T16 V0	T20 V0	T24 V0	T28 V0
1	T1 V0	T5 V0	T9 V0	T13 V0	T17 V0	T21 V0	T25 V0	T29 V0
2	T2 V0	T6 V0	T10 V0	T14 V0	T18 V0	T22 V0	T26 V0	T30 V0
3	T3 V0	T7 V0	T11 V0	T15 V0	T19 V0	T23 V0	T27 V0	T31 V0
4	T0 V1	T4 V1	T8 V1	T12 V1	T16 V1	T20 V1	T24 V1	T28 V1
5	T1 V1	T5 V1	T9 V1	T13 V1	T17 V1	T21 V1	T25 V1	T29 V1
6	T2 V1	T6 V1	T10 V1	T14 V1	T18 V1	T22 V1	T26 V1	T30 V1
7	T3 V1	T7 V1	T11 V1	T15 V1	T19 V1	T23 V1	T27 V1	T31 V1

	0	1	2	3	4	5	6	7
0	T0 V0	T1 V0	T2 V0	T3 V0	T0 V2	T1 V2	T2 V2	T3 V2
1	T4 V0	T5 V0	T6 V0	T7 V0	T4 V2	T5 V2	T6 V2	T7 V2
2	T8 V0	T9 V0	T10 V0	T11 V0	T8 V2	T9 V2	T10 V2	T11 V2
3	T12 V0	T13 V0	T14 V0	T15 V0	T12 V2	T13 V2	T14 V2	T15 V2
4	T16 V0	T17 V0	T18 V0	T19 V0	T16 V2	T17 V2	T18 V2	T19 V2
5	T20 V0	T21 V0	T22 V0	T23 V0	T20 V2	T21 V2	T22 V2	T23 V2
6	T24 V0	T25 V0	T26 V0	T27 V0	T24 V2	T25 V2	T26 V2	T27 V2
7	T28 V0	T29 V0	T30 V0	T31 V0	T28 V2	T29 V2	T30 V2	T31 V2
8	T0 V1	T1 V1	T2 V1	T3 V1	T0 V3	T1 V3	T2 V3	T3 V3
9	T4 V1	T5 V1	T6 V1	T7 V1	T4 V3	T5 V3	T6 V3	T7 V3
10	T8 V1	T9 V1	T10 V1	T11 V1	T8 V3	T9 V3	T10 V3	T11 V3
11	T12 V1	T13 V1	T14 V1	T15 V1	T12 V3	T13 V3	T14 V3	T15 V3
12	T16 V1	T17 V1	T18 V1	T19 V1	T16 V3	T17 V3	T18 V3	T19 V3
13	T20 V1	T21 V1	T22 V1	T23 V1	T20 V3	T21 V3	T22 V3	T23 V3
14	T24 V1	T25 V1	T26 V1	T27 V1	T24 V3	T25 V3	T26 V3	T27 V3
15	T28 V1	T29 V1	T30 V1	T31 V1	T28 V3	T29 V3	T30 V3	T31 V3

0	T0 V0	T0 V1	T1 V0	T1 V1	T2 V0	T2 V1	T3 V0	T3 V1
1	T4 V0	T4 V1	T5 V0	T5 V1	T6 V0	T6 V1	T7 V0	T7 V1
2	T8 V0	T8 V1	T9 V0	T9 V1	T10 V0	T10 V1	T11 V0	T11 V1
3	T12 V0	T12 V1	T13 V0	T13 V1	T14 V0	T14 V1	T15 V0	T15 V1
4	T16 V0	T16 V1	T17 V0	T17 V1	T18 V0	T18 V1	T19 V0	T19 V1
5	T20 V0	T20 V1	T21 V0	T21 V1	T22 V0	T22 V1	T23 V0	T23 V1
6	T24 V0	T24 V1	T25 V0	T25 V1	T26 V0	T26 V1	T27 V0	T27 V1
7	T28 V0	T28 V1	T29 V0	T29 V1	T30 V0	T30 V1	T31 V0	T31 V1
8	T0 V2	T0 V3	T1 V2	T1 V3	T2 V2	T2 V3	T3 V2	T3 V3
9	T4 V2	T4 V3	T5 V2	T5 V3	T6 V2	T6 V3	T7 V2	T7 V3
10	T8 V2	T8 V3	T9 V2	T9 V3	T10 V2	T10 V3	T11 V2	T11 V3
11	T12 V2	T12 V3	T13 V2	T13 V3	T14 V2	T14 V3	T15 V2	T15 V3
12	T16 V2	T16 V3	T17 V2	T17 V3	T18 V2	T18 V3	T19 V2	T19 V3
13	T20 V2	T20 V3	T21 V2	T21 V3	T22 V2	T22 V3	T23 V2	T23 V3
14	T24 V2	T24 V3	T25 V2	T25 V3	T26 V2	T26 V3	T27 V2	T27 V3
15	T28 V2	T28 V3	T29 V2	T29 V3	T30 V2	T30 V3	T31 V2	T31 V3

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	T0 V0	T4 V0	T8 V0	T12 V0	T16 V0	T20 V0	T24 V0	T28 V0	T32 V0	T36 V0	T40 V0	T44 V0	T48 V0	T52 V0	T56 V0	T60 V0
1	T1 V0	T5 V0	T9 V0	T13 V0	T17 V0	T21 V0	T25 V0	T29 V0	T33 V0	T37 V0	T41 V0	T45 V0	T49 V0	T53 V0	T57 V0	T61 V0
2	T2 V0	T6 V0	T10 V0	T14 V0	T18 V0	T22 V0	T26 V0	T30 V0	T34 V0	T38 V0	T42 V0	T46 V0	T50 V0	T54 V0	T58 V0	T62 V0
3	T3 V0	T7 V0	T11 V0	T15 V0	T19 V0	T23 V0	T27 V0	T31 V0	T35 V0	T39 V0	T43 V0	T47 V0	T51 V0	T55 V0	T59 V0	T63 V0
4	T0 V1	T4 V1	T8 V1	T12 V1	T16 V1	T20 V1	T24 V1	T28 V1	T32 V1	T36 V1	T40 V1	T44 V1	T48 V1	T52 V1	T56 V1	T60 V1
5	T1 V1	T5 V1	T9 V1	T13 V1	T17 V1	T21 V1	T25 V1	T29 V1	T33 V1	T37 V1	T41 V1	T45 V1	T49 V1	T53 V1	T57 V1	T61 V1
6	T2 V1	T6 V1	T10 V1	T14 V1	T18 V1	T22 V1	T26 V1	T30 V1	T34 V1	T38 V1	T42 V1	T46 V1	T50 V1	T54 V1	T58 V1	T62 V1
7	T3 V1	T7 V1	T11 V1	T15 V1	T19 V1	T23 V1	T27 V1	T31 V1	T35 V1	T39 V1	T43 V1	T47 V1	T51 V1	T55 V1	T59 V1	T63 V1

0 1 2 3 4 5 6 7

0	T0 V0	T1 V0	T2 V0	T3 V0	T0 V1	T1 V1	T2 V1	T3 V1
1	T4 V0	T5 V0	T6 V0	T7 V0	T4 V1	T5 V1	T6 V1	T7 V1
2	T8 V0	T9 V0	T10 V0	T11 V0	T8 V1	T9 V1	T10 V1	T11 V1
3	T12 V0	T13 V0	T14 V0	T15 V0	T12 V1	T13 V1	T14 V1	T15 V1
4	T16 V0	T17 V0	T18 V0	T19 V0	T16 V1	T17 V1	T18 V1	T19 V1
5	T20 V0	T21 V0	T22 V0	T23 V0	T20 V1	T21 V1	T22 V1	T23 V1
6	T24 V0	T25 V0	T26 V0	T27 V0	T24 V1	T25 V1	T26 V1	T27 V1
7	T28 V0	T29 V0	T30 V0	T31 V0	T28 V1	T29 V1	T30 V1	T31 V1
8	T0 V1	T1 V1	T2 V1	T3 V1	T0 V2	T1 V2	T2 V2	T3 V2
9	T4 V1	T5 V1	T6 V1	T7 V1	T4 V2	T5 V2	T6 V2	T7 V2
10	T8 V1	T9 V1	T10 V1	T11 V1	T8 V2	T9 V2	T10 V2	T11 V2
11	T12 V1	T13 V1	T14 V1	T15 V1	T12 V2	T13 V2	T14 V2	T15 V2
12	T16 V1	T17 V1	T18 V1	T19 V1	T16 V2	T17 V2	T18 V2	T19 V2
13	T20 V1	T21 V1	T22 V1	T23 V1	T20 V2	T21 V2	T22 V2	T23 V2
14	T24 V1	T25 V1	T26 V1	T27 V1	T24 V2	T25 V2	T26 V2	T27 V2
15	T28 V1	T29 V1	T30 V1	T31 V1	T28 V2	T29 V2	T30 V2	T31 V2
16	T32 V0	T33 V0	T34 V0	T35 V0	T32 V1	T33 V1	T34 V1	T35 V1
17	T36 V0	T37 V0	T38 V0	T39 V0	T36 V1	T37 V1	T38 V1	T39 V1
18	T40 V0	T41 V0	T42 V0	T43 V0	T40 V1	T41 V1	T42 V1	T43 V1
19	T44 V0	T45 V0	T46 V0	T47 V0	T44 V1	T45 V1	T46 V1	T47 V1
20	T48 V0	T49 V0	T50 V0	T51 V0	T48 V1	T49 V1	T50 V1	T51 V1
21	T52 V0	T53 V0	T54 V0	T55 V0	T52 V1	T53 V1	T54 V1	T55 V1
22	T56 V0	T57 V0	T58 V0	T59 V0	T56 V1	T57 V1	T58 V1	T59 V1
23	T60 V0	T61 V0	T62 V0	T63 V0	T60 V1	T61 V1	T62 V1	T63 V1
24	T32 V1	T33 V1	T34 V1	T35 V1	T32 V2	T33 V2	T34 V2	T35 V2
25	T36 V1	T37 V1	T38 V1	T39 V1	T36 V2	T37 V2	T38 V2	T39 V2
26	T40 V1	T41 V1	T42 V1	T43 V1	T40 V2	T41 V2	T42 V2	T43 V2
27	T44 V1	T45 V1	T46 V1	T47 V1	T44 V2	T45 V2	T46 V2	T47 V2
28	T48 V1	T49 V1	T50 V1	T51 V1	T48 V2	T49 V2	T50 V2	T51 V2
29	T52 V1	T53 V1	T54 V1	T55 V1	T52 V2	T53 V2	T54 V2	T55 V2
30	T56 V1	T57 V1	T58 V1	T59 V1	T56 V2	T57 V2	T58 V2	T59 V2
31	T60 V1	T61 V1	T62 V1	T63 V1	T60 V2	T61 V2	T62 V2	T63 V2

0	T0 V0	T1 V0	T2 V0	T3 V0	T4 V0	T5 V0	T6 V0	T7 V0	T8 V0	T9 V0	T10 V0	T11 V0	T12 V0	T13 V0	T14 V0	T15 V0
1	T16 V0	T17 V0	T18 V0	T19 V0	T20 V0	T21 V0	T22 V0	T23 V0	T24 V0	T25 V0	T26 V0	T27 V0	T28 V0	T29 V0	T30 V0	T31 V0
2	T32 V0	T33 V0	T34 V0	T35 V0	T36 V0	T37 V0	T38 V0	T39 V0	T40 V0	T41 V0	T42 V0	T43 V0	T44 V0	T45 V0	T46 V0	T47 V0
3	T48 V0	T49 V0	T50 V0	T51 V0	T52 V0	T53 V0	T54 V0	T55 V0	T56 V0	T57 V0	T58 V0	T59 V0	T60 V0	T61 V0	T62 V0	T63 V0
4	T64 V0	T65 V0	T66 V0	T67 V0	T68 V0	T69 V0	T70 V0	T71 V0	T72 V0	T73 V0	T74 V0	T75 V0	T76 V0	T77 V0	T78 V0	T79 V0
5	T80 V0	T81 V0	T82 V0	T83 V0	T84 V0	T85 V0	T86 V0	T87 V0	T88 V0	T89 V0	T90 V0	T91 V0	T92 V0	T93 V0	T94 V0	T95 V0
6	T96 V0	T97 V0	T98 V0	T99 V0	T100 V0	T101 V0	T102 V0	T103 V0	T104 V0	T105 V0	T106 V0	T107 V0	T108 V0	T109 V0	T110 V0	T111 V0
7	T112 V0	T113 V0	T114 V0	T115 V0	T116 V0	T117 V0	T118 V0	T119 V0	T120 V0	T121 V0	T122 V0	T123 V0	T124 V0	T125 V0	T126 V0	T127 V0
8	T128 V0	T129 V0	T130 V0	T131 V0	T132 V0	T133 V0	T134 V0	T135 V0	T136 V0	T137 V0	T138 V0	T139 V0	T140 V0	T141 V0	T142 V0	T143 V0
9	T144 V0	T145 V0	T146 V0	T147 V0	T148 V0	T149 V0	T150 V0	T151 V0	T152 V0	T153 V0	T154 V0	T155 V0	T156 V0	T157 V0	T158 V0	T159 V0
10	T160 V0	T161 V0	T162 V0	T163 V0	T164 V0	T165 V0	T166 V0	T167 V0	T168 V0	T169 V0	T170 V0	T171 V0	T172 V0	T173 V0	T174 V0	T175 V0
11	T176 V0	T177 V0	T178 V0	T179 V0	T180 V0	T181 V0	T182 V0	T183 V0	T184 V0	T185 V0	T186 V0	T187 V0	T188 V0	T189 V0	T190 V0	T191 V0
12	T192 V0	T193 V0	T194 V0	T195 V0	T196 V0	T197 V0	T198 V0	T199 V0	T200 V0	T201 V0	T202 V0	T203 V0	T204 V0	T205 V0	T206 V0	T207 V0
13	T208 V0	T209 V0	T210 V0	T211 V0	T212 V0	T213 V0	T214 V0	T215 V0	T216 V0	T217 V0	T218 V0	T219 V0	T220 V0	T221 V0	T222 V0	T223 V0
14	T224 V0	T225 V0	T226 V0	T227 V0	T228 V0	T229 V0	T230 V0	T231 V0	T232 V0	T233 V0	T234 V0	T235 V0	T236 V0	T237 V0	T238 V0	T239 V0
15	T240 V0	T241 V0	T242 V0	T243 V0	T244 V0	T245 V0	T246 V0	T247 V0	T248 V0	T249 V0	T250 V0	T251 V0	T252 V0	T253 V0	T254 V0	T255 V0
16	T256 V0	T257 V0	T258 V0	T259 V0	T260 V0	T261 V0	T262 V0	T263 V0	T264 V0	T265 V0	T266 V0	T267 V0	T268 V0	T269 V0	T270 V0	T271 V0
17	T272 V0	T273 V0	T274 V0	T275 V0	T276 V0	T277 V0	T278 V0	T279 V0	T280 V0	T281 V0	T282 V0	T283 V0	T284 V0	T285 V0	T286 V0	T287 V0
18	T288 V0	T289 V0	T290 V0	T291 V0	T292 V0	T293 V0	T294 V0	T295 V0	T296 V0	T297 V0	T298 V0	T299 V0	T300 V0	T301 V0	T302 V0	T303 V0
19	T304 V0	T305 V0	T306 V0	T307 V0	T308 V0	T309 V0	T310 V0	T311 V0	T312 V0	T313 V0	T314 V0	T315 V0	T316 V0	T317 V0	T318 V0	T319 V0
20	T320 V0	T321 V0	T322 V0	T323 V0	T324 V0	T325 V0	T326 V0	T327 V0	T328 V0	T329 V0	T330 V0	T331 V0	T332 V0	T333 V0	T334 V0	T335 V0
21	T336 V0	T337 V0	T338 V0	T339 V0	T340 V0	T341 V0	T342 V0	T343 V0	T344 V0	T345 V0	T346 V0	T347 V0	T348 V0	T349 V0	T350 V0	T351 V0
22	T352 V0	T353 V0	T354 V0	T355 V0	T356 V0	T357 V0	T358 V0	T359 V0	T360 V0	T361 V0	T362 V0	T363 V0	T364 V0	T365 V0	T366 V0	T367 V0
23	T368 V0	T369 V0	T370 V0	T371 V0	T372 V0	T373 V0	T374 V0	T375 V0	T376 V0	T377 V0	T378 V0	T379 V0	T380 V0	T381 V0	T382 V0	T383 V0
24	T384 V0	T385 V0	T386 V0	T387 V0	T388 V0	T389 V0	T390 V0	T391 V0	T392 V0	T393 V0	T394 V0	T395 V0	T396 V0	T397 V0	T398 V0	T399 V0
25	T400 V0	T401 V0	T402 V0	T403 V0	T404 V0	T405 V0	T406 V0	T407 V0	T408 V0	T409 V0	T410 V0	T411 V0	T412 V0	T413 V0	T414 V0	T415 V0
26	T416 V0	T417 V0	T418 V0	T419 V0	T420 V0	T421 V0	T422 V0	T423 V0	T424 V0	T425 V0	T426 V0	T427 V0	T428 V0	T429 V0	T430 V0	T431 V0
27	T432 V0	T433 V0	T434 V0	T435 V0	T436 V0	T437 V0	T438 V0	T439 V0	T440 V0	T441 V0	T442 V0	T443 V0	T444 V0	T445 V0	T446 V0	T447 V0
28	T448 V0	T449 V0	T450 V0	T451 V0	T452 V0	T453 V0	T454 V0	T455 V0	T456 V0	T457 V0	T458 V0	T459 V0	T460 V0	T461 V0	T462 V0	T463 V0
29	T464 V0	T465 V0	T466 V0	T467 V0	T468 V0	T469 V0	T470 V0	T471 V0	T472 V0	T473 V0	T474 V0	T475 V0	T476 V0	T477 V0	T478 V0	T479 V0
30	T480 V0	T481 V0	T482 V0	T483 V0	T484 V0	T485 V0	T486 V0	T487 V0	T488 V0	T489 V0	T490 V0	T491 V0	T492 V0	T493 V0	T494 V0	T495 V0
31	T496 V0	T497 V0	T498 V0	T499 V0	T500 V0	T501 V0	T502 V0	T503 V0	T504 V0	T505 V0	T506 V0	T507 V0	T508 V0	T509 V0	T510 V0	T511 V0

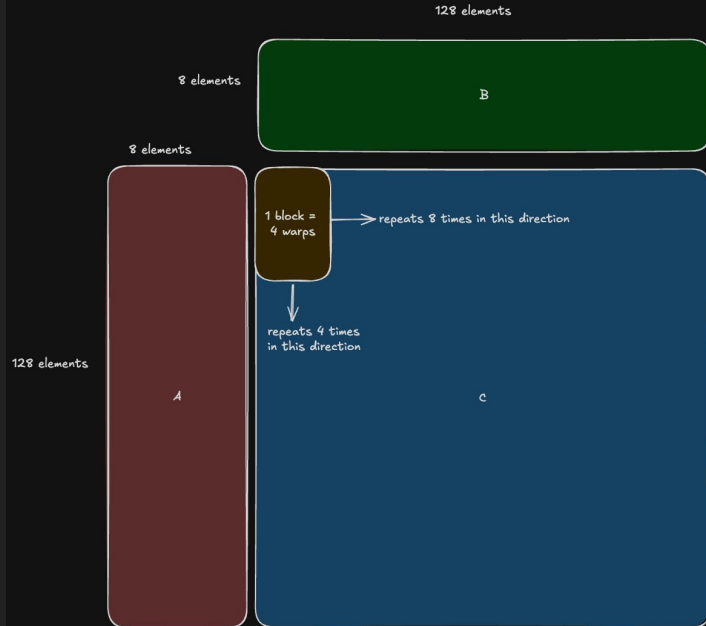

```
// CuTe simple MMA kernel (TF32, no shared memory)
if (should_run("CuTe Simple MMA", kernel_filters)) {
    using tiled_mma_t = TiledMMA<
        MMA_Atom<SM80_16x8x8_F32TF32F32FN>,
        Layout<Shape<_2, _2, _1>> // 4 warps = 128 threads
    >;

    constexpr int BLK_M = 128, BLK_N = 128, BLK_K = 8;
    tiled_mma_t tiled_mma;

    dim3 block_cute(size(tiled_mma));
    dim3 grid_cute(M / BLK_M, N / BLK_N);

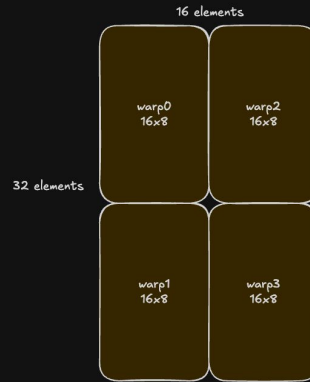
    auto cute_op = [g]() {
        matmul_cute_simple<BLK_M, BLK_N, BLK_K>
            <<<grid_cute, block_cute>>>(d_A, d_B, d_C, M, K, N, tiled_mma);
    };

    run_benchmark("CuTe Simple MMA", cute_op, d_C, h_C, h_A, h_B, M, K, N, iterations, benchmark_model);
}
```



One thread block (of 128 threads) handles a 128x128 block of C

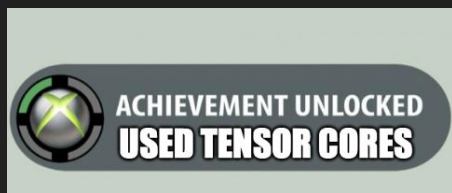
Shape<_2, _2, _1> (column major by default)



```
440 // Main loop over K
441 int k_tile_max = size<2>(gA);
442 for (int k = 0; k < k_tile_max; k++) {
443     copy(thr_mma.partition_A(gA(_ , _ , k)), tCrA);
444     copy(thr_mma.partition_B(gB(_ , _ , k)), tCrB);
445     gemm(tiled_mma, tCrA, tCrB, tCrC); Pin select
446 }
447 // Store result
448 copy(tCrC, tCgC);
449 }
450
```

This "gemm" call calls the MMA atom 8x4=32 times

Kernel	Duration (ms)	TFLOP/S	%age of cublas FP32	%age of cublas TF32
cuBLAS (FP32)	30.96	35.51	100%	11%
cuBLAS (TF32)	3.46	317.78	895%	100%
matmul_naive	296.24	3.71	10%	1%
matmul_tiled<32>	176.78	6.22	18%	2%
matmul_tiled_reg	67.7	16.24	46%	5%
matmul_tiled_vec	50.42	21.81	61%	7%
matmul_tiled_pipe	73.32	15	42%	5%
matmul_cute_simple	25.9	42.45	120%	13%



matmul_cute_smem

- Tiling similar to earlier saves GMEM bandwidth
- async fetching to bypass the L1 cache
- GMEM -> SMEM -> REGS -> MMA

matmul_cute_smem

```
// CuTe SMEM kernel: global -> shared memory (cp.async) -> registers -> MMA

template <int BLK_M, int BLK_N, int BLK_K,
         class TiledMMA, class CopyA, class CopyB,
         class SmemLayoutA, class SmemLayoutB>
__global__ void matmul_cute_smem(const float* A_ptr, const float* B_ptr, float* C_ptr,
                                int M, int K, int N,
                                TiledMMA tiled_mma, CopyA copy_a, CopyB copy_b,
                                SmemLayoutA sA_layout, SmemLayoutB sB_layout) {

    extern __shared__ float smem[];

    int k_tile_max = size<2>(gA);
    for (int k = 0; k < k_tile_max; k++) {
        copy(copy_a, tAgA(_, _, k), tAsA);
        copy(copy_b, tBgB(_, _, k), tBsB);
        cp_async_fence();
        cp_async_wait<0>();
        __syncthreads();

        copy(tCsA, tCrA);
        copy(tCsB, tCrB);
        gemm(tiled_mma, tCrA, tCrB, tCrC);
        __syncthreads();
    }

    copy(tCrC, tCgC);
}
```

Kernel	Duration (ms)	TFLOP/S	%age of cublas FP32	%age of cublas TF32
cuBLAS (FP32)	30.96	35.51	100%	11%
cuBLAS (TF32)	3.46	317.78	895%	100%
matmul_naive	296.24	3.71	10%	1%
matmul_tiled<32>	176.78	6.22	18%	2%
matmul_tiled_reg	67.7	16.24	46%	5%
matmul_tiled_vec	50.42	21.81	61%	7%
matmul_tiled_pipe	73.32	15	42%	5%
matmul_cute_simple	25.9	42.45	120%	13%
matmul_cute_smem (k=8)	17.85	61.6	173%	19%
matmul_cute_smem (k=32)	11.53	95.36	269%	30%

matmul_cute_wgmma

- WGMMMA for 4-warp group MMA
 - Directly operates from SMEM
 - Async MMA (code can do index math and bookkeeping concurrently with matmul)
 - Reduces instruction issue overhead because of larger tile
- GMEM -> SMEM -> WGMMMA

matmul_cute_wgmma

```
// CuTe SM90 wgmma kernel: global -> swizzled shared memory (cp.async) -> wgmma.
// wgmma reads A and B directly from shared memory via descriptors,
// eliminating the smem->register copy. Requires K-major swizzled smem layouts
// and a pre-transposed B matrix (K contiguous).
template <int BLK_M, int BLK_N, int BLK_K,
          class TiledMMA, class CopyA, class CopyB,
          class SmemLayoutA, class SmemLayoutB>
__global__ void matmul_cute_wgmma(const float* A_ptr, const float* B_T_ptr, float* C_ptr,
                                  int M, int K, int N,
                                  TiledMMA tiled_mma, CopyA copy_a, CopyB copy_b,
                                  SmemLayoutA sA_layout, SmemLayoutB sB_layout) {

    extern __shared__ float smem[];

    clear(tCrC);

    int k_tile_max = size<2>(gA);
    for (int k = 0; k < k_tile_max; k++) {
        // Load A and B tiles into swizzled smem via cp.async
        copy(copy_a, tAgA(_, _, k), tAsA);
        copy(copy_b, tBgB(_, _, k), tBsB);
        cp_async_fence();
        cp_async_wait<0>();
        __syncthreads();

        // wgmma: reads A, B from smem descriptors; accumulates in registers
        warpgroup_fence_operand(tCrC);
        warpgroup_arrive();
        gemm(tiled_mma, tCsA, tCsB, tCrC);
        warpgroup_commit_batch();
        warpgroup_wait<0>();
        warpgroup_fence_operand(tCrC);
        __syncthreads();
    }

    copy(tCrC, tCgC);
}
```

Kernel	Duration (ms)	TFLOP/S	%age of cublas FP32	%age of cublas TF32
cuBLAS (FP32)	30.96	35.51	100%	11%
cuBLAS (TF32)	3.46	317.78	895%	100%
matmul_naive	296.24	3.71	10%	1%
matmul_tiled<32>	176.78	6.22	18%	2%
matmul_tiled_reg	67.7	16.24	46%	5%
matmul_tiled_vec	50.42	21.81	61%	7%
matmul_tiled_pipe	73.32	15	42%	5%
matmul_cute_simple	25.9	42.45	120%	13%
matmul_cute_smem (k=8)	17.85	61.6	173%	19%
matmul_cute_smem (k=32)	11.53	95.36	269%	30%
matmul_cute_persistent	21.24	51.77	146%	16%
matmul_cute_swizzle	14.27	77.05	217%	24%
matmul_cute_pipe	31.78	34.6	97%	11%
matmul_cute_wgmma	6.52	168.64	475%	53%

matmul_cute_wgmma_tma

- Uses TMA instead of cp.async
 - fewer instruction issue
 - bounds checking in HW
 - HW swizzling
 - Multi-casting (not done yet)
- Use HW memory barriers
- Pipelining

matmul_cute_wgmma_tma

```
// CuTe SM90 wgmma + TMA kernel.
// TMA loads data from global -> swizzled smem using a single thread.
// wgmma reads A,B from smem via descriptors. Multi-stage pipeline with
// barrier-based producer-consumer synchronization.
template <class CtaTiler,
          class TmaA, class TmaB,
          class SmemLayoutA, class SmemLayoutB,
          class TiledMMA>
__global__ static
__launch_bounds__(decltype(size(TiledMMA{}))::value)
void matmul_cute_wgmma_tma(int M, int K, int N,
                          CUTLASS_GRID_CONSTANT TmaA const tma_a,
                          CUTLASS_GRID_CONSTANT TmaB const tma_b,
                          float* C_ptr, TiledMMA tiled_mma) {
    while (k_tile_count > -K_PIPE_MAX) {
        // Wait for TMA producer to complete this stage

        // wgmma on this stage
        gemm(tiled_mma, tCrA(_, _, _, read_pipe), tCrB(_, _, _, read_pipe), tCrC);

        // Signal consumption done

        // Issue next TMA load (only thread 0)
        if ((warp_idx == 0) && lane_predicate && (k_tile_count > 0)) {
            copy(tma_a.with(producer_mbar[pipe]), tAgA(_, k_tile), tAsA(_, pipe));
            copy(tma_b.with(producer_mbar[pipe]), tBgB(_, k_tile), tBsB(_, pipe));
        }

        // Store result
        copy(tCrC, tCgC);
    }
}
```

Kernel	Duration (ms)	TFLOP/S	%age of cublas FP32	%age of cublas TF32
cuBLAS (FP32)	30.96	35.51	100%	11%
cuBLAS (TF32)	3.46	317.78	895%	100%
matmul_naive	296.24	3.71	10%	1%
matmul_tiled<32>	176.78	6.22	18%	2%
matmul_tiled_reg	67.7	16.24	46%	5%
matmul_tiled_vec	50.42	21.81	61%	7%
matmul_tiled_pipe	73.32	15	42%	5%
matmul_cute_simple	25.9	42.45	120%	13%
matmul_cute_smem (k=8)	17.85	61.6	173%	19%
matmul_cute_smem (k=32)	11.53	95.36	269%	30%
matmul_cute_persistent	21.24	51.77	146%	16%
matmul_cute_swizzle	14.27	77.05	217%	24%
matmul_cute_pipe	31.78	34.6	97%	11%
matmul_cute_wgmma	6.52	168.64	475%	53%
matmul_cute_wgmma_pipe	4.48	245.43	691%	77%
matmul_cute_wgmma_pipe (Swizzle)	4.44	247.64	697%	78%
matmul_cute_wgmma_tma (128x128)	4.51	243.79	687%	77%
matmul_cute_wgmma_tma (128x256)	3.55	309.72	872%	97%
CUTLASS WS GEMM	3.93	279.77	788%	88%

ID	Estimated Speedup [%]	Function Name	Demangled Name	Duration [ms] (847.59 ms)	Runtime Improvement [ms] (341.74 ms)	Compute Throughput	Memory Throughput	# Registers [register]	Grid Size	Block Size [block]	Result Type
0	42.72	matmul_naive	matmul_naive(const float *, const float *, float ...	296.22	126.56	65.10	97.65	32	512, 256, ...	16, 16, ...	Kernel: SIMT
1	16.06	matmul_tiled	void matmul_tiled<32>(const float *, const..flo...	176.78	28.38	75.00	89.33	32	256, 128, ...	32, 32, ...	Kernel: SIMT
2	47.21	matmul_tiled_reg	void matmul_tiled_reg<128.8..const float *, co...	67.69	31.96	43.28	52.79	162	64, 32, ...	256, 1, ...	Kernel: SIMT
3	64.85	matmul_tiled_vec	void matmul_tiled_vec<128.8..const float *, co...	50.12	32.50	54.00	74.12	128	64, 32, ...	256, 1, ...	Kernel: SIMT
4	50.42	matmul_tiled_pipe	void matmul_tiled_pipe<128.8..const float *, c...	73.34	36.98	37.55	49.58	130	64, 32, ...	256, 1, ...	Kernel: SIMT
5	37.50	matmul_cute_simple	..matmul_cute_simple..SM80_16x8x8_F32TF3...	25.97	9.74	18.70	74.99	166	32, 64, ...	128, 1, ...	Kernel: SIMT
6	59.31	matmul_cute_smern	..matmul_cute_smern..SM80_16x8x8_F32TF3...	17.84	10.58	27.50	40.69	188	32, 64, ...	128, 1, ...	Kernel: SIMT
7	46.31	matmul_cute_smern	..matmul_cute_smern..SM80_16x8x8_F32TF3...	11.53	5.34	42.59	53.69	190	32, 64, ...	128, 1, ...	Kernel: SIMT
8	70.71	matmul_cute_persistent	..matmul_cute_persistent..SM80_16x8x8_F32...	20.82	14.72	23.53	29.29	188	132, 1, ...	128, 1, ...	Kernel: SIMT
9	56.63	matmul_cute_swizzle	..matmul_cute_swizzle..SM80_16x8x8_F32TF...	14.25	8.07	34.39	43.37	186	2048, 1, ...	128, 1, ...	Kernel: SIMT
10	45.14	matmul_cute_pipe	..matmul_cute_pipe..SM80_16x8x8_F32TF32T...	31.78	14.35	65.32	33.20	255	32, 64, ...	128, 1, ...	Kernel: SIMT
11	43.95	matmul_cute_wgmma	..matmul_cute_wgmma..MMA_64x128x8_F32...	6.22	2.73	51.22	56.05	96	32, 64, ...	256, 1, ...	Kernel: SIMT
12	29.30	matmul_cute_wgmma_pipe	..matmul_cute_wgmma_pipe..MMA_64x128x8...	4.48	1.31	72.82	69.54	124	32, 64, ...	256, 1, ...	Kernel: SIMT
13	29.51	matmul_cute_wgmma_pipe	..matmul_cute_wgmma_pipe..MMA_64x128x8...	4.45	1.31	73.58	69.82	124	2048, 1, ...	256, 1, ...	Kernel: SIMT
14	66.85	matmul_cute_wgmma_tma	..matmul_cute_wgmma_tma..Copy_Traits..Aux...	4.21	2.82	74.92	82.05	90	32, 64, ...	256, 1, ...	Kernel: SIMT
15	50.30	matmul_cute_wgmma_tma	..matmul_cute_wgmma_tma..Copy_Traits..Aux...	3.54	1.78	92.66	73.10	90	32, 32, ...	512, 1, ...	Kernel: SIMT
16	50.38	device_kernel	..MainloopSm90TmaGmmaWarpSpecialized....	3.94	1.99	82.85	56.57	168	1, 132, ...	384, 1, ...	Kernel: SIMT
17	26.63	kernel2	..Kernel2<cutlass_80_simt_sgemm..256x128...	30.96	8.24	87.38	47.49	202	256, 4, ...	256, 1, ...	Kernel: SIMT
18	69.32	sm90_xmma_gemm_f32f32_tf32f32_f32_nn...	sm90_xmma_gemm_f32f32_tf32f32_f32_nn_n...	3.44	2.39	94.44	71.56	168	66, 2, ...	384, 1, ...	Kernel: SIMT

Summary

- Use tensor cores
- Try to keep the arithmetic hardware busy
 - More math work per warp
 - More warps in the system
- Overlap compute and loads
- Use TMA for extra speedup
- CuTe makes index math simple

Questions?

- Comments and questions to ahmadsharif@gmail.com
- github.com/ahmadsharif1

Backup